UCT Benchmark Project Setup Guide

Setting Up WSL for Windows

As developers are in a mixed OS environment, it may be useful to work under Ubuntu or some other Linux distribution to better align to the syntax and logic of Unix (macOS) based devices.

Installing WSL (Install WSL | Microsoft Learn)

- Open PowerShell and run the following:

```
wsl --install
```

- If you're experiencing unexpected crashes, you may need to update your installed WSL instance
- Ubuntu is installed by default, but you can change to a different Linux distribution of your choosing
- After installation, you can launch your distributions terminal by entering 'wsl' into PowerShell or using the shortcuts generated.

Copying Project to WSL Distribution

- After launching your distribution's terminal, make sure your .zip is stored in your current user's downloads folder or note the filepath
- You can copy the project to your virtual system by running the following:

```
cp /mnt/c/Users/{Username}/Downloads/{Project.zip} {Destination Directory}
```

- Where:
  - {Username} = Windows username
  - {Project.zip} = the name of project file downloaded to your machine
  - {Destination Directory} = Where you want to copy the project file to on your Linux distribution (you can leave blank to have it copy to your current working directory)

- Run the following to unzip your project

```
sudo apt install zip unzip unzip {Project.zip}
```

Setting Up Mini-Forge for Environment Virtualization

Currently, the OreKit Python Wrapper is only available through Anaconda (conda). You will need to have mini-forge installed in order to setup our applications virtualized environment and load it's dependencies.

Installing Mini-Forge (GitHub - conda-forge/miniforge: A conda-forge distribution.)

- Fetch and run 'Miniforge3.sh' (macOS can install through Homebrew), then create a path to 'conda'

```
wget -O Miniforge3.sh "https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-$(uname)-$(uname -m).sh"
```

```
bash Miniforge3-$(uname)-$(uname -m).sh
```

```
source "${HOME}/conda/etc/profile.d/conda.sh"
```

- During installation it will ask if you want to let conda's base environment initiate at launch, you'll probably want to say no

Importing conda Virtual Environment

- Within the project directory there should be a file called 'environment.yml' which can be used to create a virtual environment with all the dependencies needed for our application. To create a virtual environment from our environment.yml file, run:

```
conda env create -f environment.yml
```

- Once you've confirmed installation and everything is complete, you can activate the generated environment to start running code!

```
conda activate {Environment Name}
```

Where:

- {Environment Name} = Should say in the environment.yml (ie name: UCTBenchMark)

Common Issues

- When working with conda:
  - Do not install packages in base environment
  - It is best practice to install your required packages at the same time as you are creating your virtual environment
  - Our program requires Orekit >12.1
  - If your terminal is crashing when generating your virtual environment, your system might not meet the resource requirements to install astropy.